

COMPSCI 689

Lecture 10: Multilayer Perceptrons

Benjamin M. Marlin

College of Information and Computer Sciences
University of Massachusetts Amherst

Slides by Benjamin M. Marlin (marlin@cs.umass.edu).

SVC Dual QP and Kernels

Kernelized SVC Dual Quadratic Program:

$$\begin{aligned} \arg \max_{\alpha} \quad & \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N y_n y_m \alpha_n \alpha_m \mathcal{K}(\mathbf{x}_n, \mathbf{x}_m) \\ \text{s.t. } \forall n \quad & 0 \leq \alpha_n \leq C \\ & \sum_{n=1}^N \alpha_n y_n = 0 \end{aligned}$$

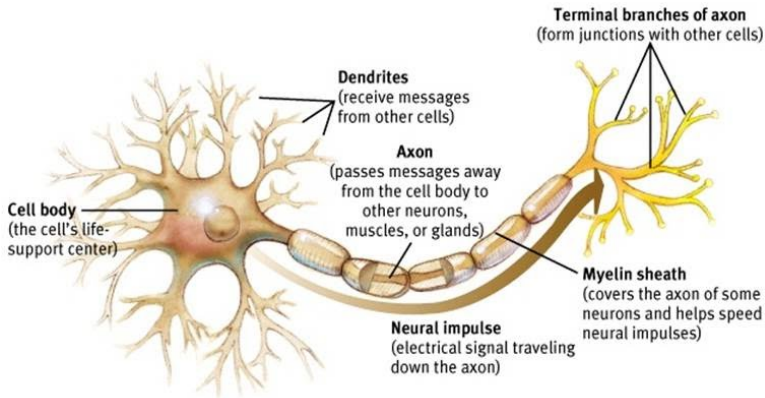
Non-Linear Kernel Functions:

- Polynomial Kernel: $\mathcal{K}(\mathbf{x}', \mathbf{x}) = (1 + \mathbf{x}' \mathbf{x}^T)^B$
- Exponential (RBF) Kernel: $\mathcal{K}(\mathbf{x}', \mathbf{x}) = a \exp \left(-b \|\mathbf{x}' - \mathbf{x}\|_2^2 \right)$

The Trouble with Basis Expansions and Kernels

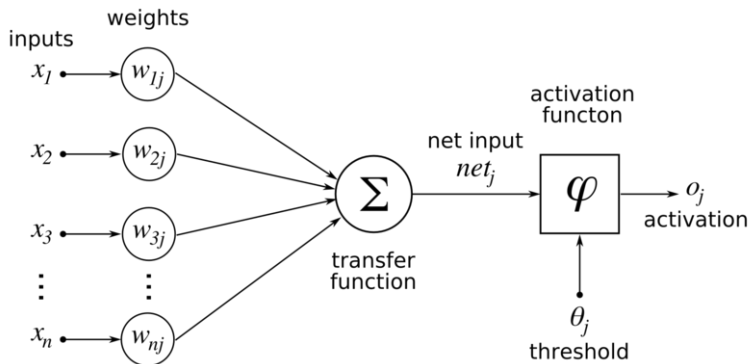
- We have previously seen how basis expansions and kernels can be used to increase the capacity of linear models, turning them into models that are non-linear in the features while remaining linear in the parameters.
- **Question:** What is the primary weakness of this approach?
- In this lecture, we'll see how neural networks have the potential to learn feature representations from data at the same time they learn to solve regression and classification problems.
- We'll start with some historical background...

Biological Neurons (Santiago Ramón y Cajal, 1888)



Map of fruit fly brain: https://flywire.ai/for_media

Single Neuron Models (McCulloch and Pitts, 1943)



Assuming $\varphi(x) = \text{sign}(x)$, what model is this?

Assuming $\varphi(x) = x$, what model is this?

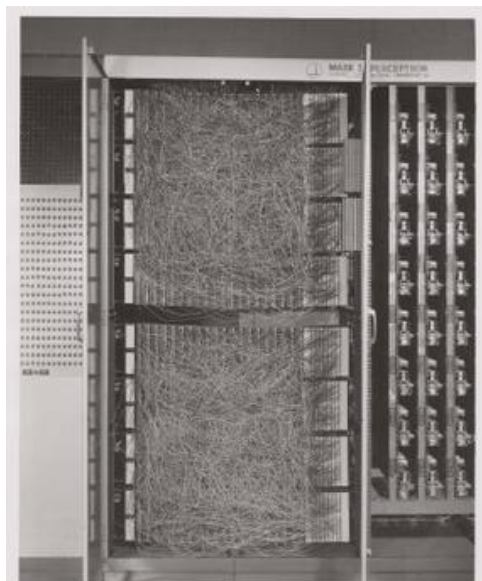
The Perceptron Algorithm (Rosenblatt, 1958)

The Perceptron Algorithm is a simple online algorithm for adapting the weights in a McCulloch/Pitts neuron with $\text{sign}()$ activation. It was developed in the 1950s by Rosenblatt at Cornell.

Algorithm PERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $w_d \leftarrow 0$ , for all  $d = 1 \dots D$  // initialize weights
2:  $b \leftarrow 0$  // initialize bias
3: for  $iter = 1 \dots MaxIter$  do
4:   for all  $(x, y) \in \mathbf{D}$  do
5:      $a \leftarrow \sum_{d=1}^D w_d x_d + b$  // compute activation for this example
6:     if  $ya \leq 0$  then
7:        $w_d \leftarrow w_d + yx_d$ , for all  $d = 1 \dots D$  // update weights
8:        $b \leftarrow b + y$  // update bias
9:     end if
10:  end for
11: end for
12: return  $w_0, w_1, \dots, w_D, b$ 
```

The Mark 1 Perceptron (Rosenblatt, 1958)



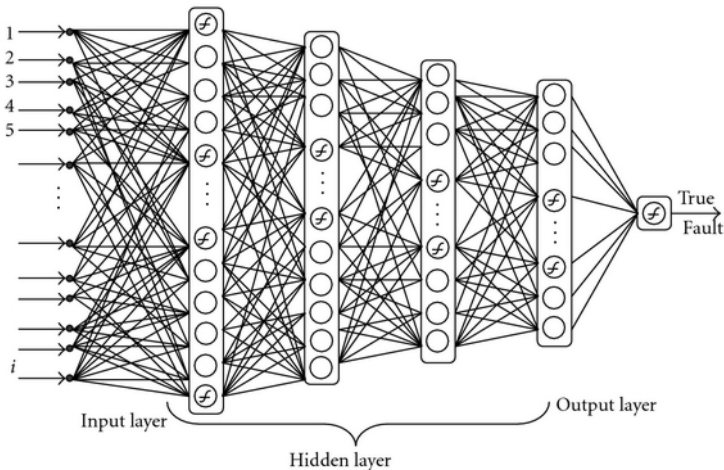
Perceptron Learning Limitations

- This perceptron algorithm converges to a random separating hyperplane when the data are linearly separable.
- However, the algorithm is not derived from optimizing an objective function and does not converge when data are not linearly separable.
- ERM/RRM for linear models typically gives classification results with improved generalization performance compared to this approach in the separable case, and can also handle the non-separable case correctly.

Perceptron Representational Limitations

- In 1969, Minsky and Papert at MIT popularized a set of arguments showing that the single-layer perceptron could not learn certain classes of functions (including XOR).
- However, Minsky and Papert also showed that more complex functions could be represented using a *multi-layer perceptron* or MLP.
- Unfortunately, at the time, no algorithms were known that could learn such networks from data.

Multi-Layer Perceptron



Sigmoid Neural Networks (1980s)

The eventual solution to MLP learning for classification was to:

- 1 Make the hidden layer non-linearities smooth (sigmoid/logistic) functions.
- 2 Make the output layer non-linearity a differentiable function.
- 3 Use a differentiable loss function.
- 4 Learn using the *backpropagation algorithm*, which was popularized by Rumelhart, Hinton and Williams in the 1980s (backprop is just vanilla gradient descent on the ERM objective defined by the loss).

Example: 1-Hidden Layer Sigmoid NN Classifier

- The simplest binary classification architecture of this type is a 1-hidden layer sigmoid neural network:

$$g_{\theta}(\mathbf{x}) = \phi_{\theta}(\mathbf{x})\theta^o$$

$$\phi_{\theta}(\mathbf{x})[k] = \sigma(\mathbf{x}\theta_k^1) = \frac{1}{1 + \exp(-(\mathbf{x}\theta_k^1))}$$

- We assume K hidden units $h_k = \phi_{\theta}(\mathbf{x})[k]$ and D input features. h_k is the value of the k^{th} hidden unit. $g_{\theta}(\mathbf{x})$ is the discriminant value output by the network. The predicted class is $\hat{y} = \text{sign}(g_{\theta}(\mathbf{x}))$.
- The parameters of the model are a set of weights θ^l in each layer l . We assume bias absorption for the inputs and a bias hidden unit.

Binary Sigmoid NN Learning Problem

- Binary Sigmoid NN Classifiers can be learned in the ERM framework using the logistic loss (assuming $y \in \{-1, 1\}$):
 $L_{\log}(y, g_{\theta}(\mathbf{x})) = \log(1 + \exp(-yg_{\theta}(\mathbf{x})))$.
- For the 1-hidden layer sigmoid neural network model with parameters $\theta = [\theta_1^1, \dots, \theta_K^1, \theta^o]$, the learning problem is to minimize $R(\theta, \mathcal{D})$ with respect to θ :

$$R(\theta, \mathcal{D}) = \sum_{n=1}^N \log(1 + \exp(-y_n g_{\theta}(\mathbf{x}_n)))$$

$$g_{\theta}(\mathbf{x}_n) = \phi_{\theta}(\mathbf{x}_n) \theta^o$$

$$\phi_{\theta}(\mathbf{x}_n)[k] = \sigma(\mathbf{x}_n \theta_k^1)$$

- Where $\sigma(x) = 1/(1 + \exp(-x))$.

Binary Sigmoid NN Classifier

- This model can be viewed as simultaneously learning a basis expansion and a logistic regression classifier in this new basis.
- The output layer is simply a hyperplane classifier applied to the basis expansion $\phi_{\theta}(\mathbf{x}_n)$.
- However, the basis expansion $\phi_{\theta}(\mathbf{x}_n)$ is itself a parametric function with parameters that can be learned from data.

Binary Sigmoid NN Learning Problem

The objective function for the learning problem is as shown below:

$$R(\theta, \mathcal{D}) = \sum_{n=1}^N \log(1 + \exp(-y_n g_{\theta}(\mathbf{x}_n)))$$

$$g_{\theta}(\mathbf{x}_n) = \phi_{\theta}(\mathbf{x}_n) \theta^o$$

$$\phi_{\theta}(\mathbf{x}_n)[k] = \sigma(\mathbf{x}_n \theta_k^1)$$

Where $\sigma(x) = 1/(1 + \exp(-x))$.

Binary Sigmoid NN Learning Problem

The objective function for the learning problem is as shown below:

$$R(\theta, \mathcal{D}) = \sum_{n=1}^N \log(1 + \exp(-y_n g_n))$$
$$g_n = \mathbf{h}_n \theta^o$$
$$\mathbf{h}_n = [\sigma(\mathbf{x}_n \theta_1^1), \dots, \sigma(\mathbf{x}_n \theta_K^1)]$$

Here, g_n and the $\mathbf{h}_n$ vector are intermediate outputs of the network. The $\mathbf{h}_n$'s correspond to the values of the hidden units.

Gradients for Binary Sigmoid NN Classifiers

- $\nabla_{\theta^o} R(\theta, \mathcal{D}) = \sum_{n=1}^N \frac{\partial R(\theta, \mathcal{D})}{\partial g_n} \nabla_{\theta^o}(g_n)$
- $\frac{\partial R(\theta, \mathcal{D})}{\partial g_n} = \frac{1}{1 + \exp(-y_n g_n)} \exp(-y_n g_n)(-y_n) = -(1 - \sigma(y_n g_n))y_n$
- $\nabla_{\theta^o}(g_n) = \nabla_{\theta^o}(\mathbf{h}_n \theta^o) = \mathbf{h}_n^T$
- $\nabla_{\theta^o} R(\theta, \mathcal{D}) = \sum_{n=1}^N (\sigma(y_n g_n) - 1)y_n \mathbf{h}_n^T$

Gradients for Binary Sigmoid NN Classifiers

- $\nabla_{\theta_k^1} R(\theta, \mathcal{D}) = \sum_{n=1}^N \frac{\partial R(\theta, \mathcal{D})}{\partial g_n} \frac{\partial g_n}{\partial \mathbf{h}_{nk}} (\nabla_{\theta_k^1} \mathbf{h}_{nk})$
- $\frac{\partial R(\theta, \mathcal{D})}{\partial g_n} = \frac{1}{1 + \exp(-y_n g_n)} \exp(-y_n g_n) (-y_n) = -(1 - \sigma(y_n g_n)) y_n$
- $\frac{\partial g_n}{\partial \mathbf{h}_{nk}} = \frac{\partial g_n}{\partial \mathbf{h}_{nk}} (\mathbf{h}_n \theta_k^o) = \theta_k^o$
- $\nabla_{\theta_k^1} (\mathbf{h}_{nk}) = \nabla_{\theta_k^1} \sigma(\mathbf{x}_n \theta_k^1) = \sigma'(\mathbf{x}_n \theta_k^1) \mathbf{x}_n^T = h_{nk} (1 - h_{nk}) \mathbf{x}_n^T$
- $\nabla_{\theta_k^1} R(\theta, \mathcal{D}) = \sum_{n=1}^N (\sigma(y_n g_n) - 1) y_n \theta_k^o h_{nk} (1 - h_{nk}) \mathbf{x}_n^T$
- Noting that $\sigma'(x) = \sigma(x)(1 - \sigma(x))$

Learning for Sigmoid NN Regression

- Once we have the gradient, we can apply any gradient-based optimization algorithm
- Learning sigmoid neural networks for regression follows the same basic pattern.
- All we need to do is switch to a regression loss. The model defined so far already has a linear output layer.
- The derivation of the gradient is nearly identical regardless of the loss used.

Regularization

- Like with linear regression, logistic regression, and SVMs, regularizing neural networks typically improves generalization performance.
- Standard regularizers can be added to the objective function to accomplish this. (i.e. $+\lambda\|\mathbf{w}\|_2^2$).
- In the neural networks literature, the addition of two-norm regularization is sometimes referred to as *weight decay* because of the contribution to the learning step (i.e., $-\lambda\mathbf{w}$).
- The number of layers and the width of the layers are also both important architectural hyper-parameters that impact model capacity.

Sigmoid Neural Network Limitations

- Unlike SVMs, logistic regression and OLS linear regression, multi-layer sigmoid neural network models have many local optima.
- In practice, it can be hard to find a combination of layer sizes and regularization that yields good generalization performance when learning from small or moderate amounts of data.
- In addition, the use of sigmoid functions for the hidden units results in a *vanishing gradient* problem that can further slow down learning for deep networks.
- As a result, there were few examples of deep neural network models that resulted in performance that exceeded hand-crafted features, basis expansions and kernels until the last decade.

Perceptrons to Deep Learning

- 1 Address the vanishing gradient problem using rectified linear units: $relu(x) = \max(0, x)$.
- 2 Have access to lots of labeled data (ie: millions of examples).
- 3 Do the computing on GPUs to achieve significant speedups (i.e.: model training takes days instead of weeks or months).
- 4 The breakthrough paper in this area is Krizhevsky, Sutskever, and Hinton. *Imagenet classification with deep convolutional neural networks*. Advances in neural information processing systems 25, 2012.